Technical Interview Guide: Candidate Trust at the Invite Boundary

Purpose: A line-by-line, decision-by-decision walkthrough of the PR so you can explain *what* you did, *why* you did it, and *how* it works. Use this as your cheat-sheet during the interview.

1. The Slice in One Sentence

> **"Invite lifecycle + tenant security"** — make interview links expire, revocable, re-issuable, and honest about their state; record explicit consent; and ensure an assessor can never touch another tenant's portfolio.

Why this slice? Highest leverage for a one-day deadline: touches auth, privacy, candidate trust, assessor workflow, and is 100 % demo-able without a Gemini key.

2. Problem → Fix Map (The "Before vs After")

Area	Baseline (Broken)	What I Shipped
Invite URL	Built with `APP_BASE_URL` (API origin) → 404	`WEB_BASE_URL` / `CANDIDATE_BASE_URL` → React route `/interview/:token`
Expiry	None — links lived forever	`invite_expires_at` (default 72 h, configurable via `INVITE_TTL_HOURS`)
Revocation	Impossible	`invite_revoked_at`; only allowed on *pending* invites (422 for live/completed)
Reissue	N/A	**Token rotation** — new token, new expiry, old link stays dead, `invite_reissued_count++`
Candidate states	Every error → "Interview Complete"	6 distinct states + stable API codes: `invite_not_found` (404), `invite_expired` (410), `invite_revoked` (410),

		`invite_completed` (410), `invite_in_progress` (410), `network` (0/5xx)
Consent	Missing — mic opened immediately	Affirmative checkbox, server-recorded `consent_given_at`, idempotent `POST /sessions/:token/consent`, EN/ID copy
Tenant isolation (portfolio)	`Portfolio.find(id)` — IDOR	Lookups join through `sessions` + filter `sessions.tenant_id = current_tenant_id`; cross-tenant → 404
Tenant isolation (override)	`PortfolioSkill.find(id)` — IDOR	Same pattern
Assessor role	`ASSESSOR_ROLES = %w[admin]`; `ROLES` missing `assessor`	`ASSESSOR_ROLES = %w[admin assessor]`; `ROLES << 'assessor'`; login accepts both
Assessor UI	Single "Awaiting/Live" label; no actions for dead links	Status chips, expiry, copy/revoke for pending, reissue for expired/revoked, monitor for live, results for completed; live-only polling
Tests	No runner, no CI	43 RSpec + 24 Vitest; CI on every push/PR; seeded-fault proof
Seeded demo	None	`db/seeds_demo.rb` → 6 candidates covering all states + portfolio/fit-gap

3. Backend Deep Dive

3.1 Migration

```
add_column :sessions, :invite_expires_at, :datetime
add_column :sessions, :invite_revoked_at, :datetime
add_column :sessions, :consent_given_at, :datetime
add_column :sessions, :invite_reissued_count, :integer, default: 0, null: false

# Backfill: give existing links the same 72 h window from created_at
execute <<~SQL
  UPDATE ai_interview.sessions
  SET invite_expires_at = created_at + INTERVAL '72 hours'
  WHERE invite_expires_at IS NULL
SQL

change_column_null :sessions, :invite_expires_at, false
```

Why additive + reversible? Zero-risk deploy. No backfill service, no dual-write. `down` simply drops the four columns.

Why 72 h default? Defensible "business hours × 3" window; configurable via `INVITE_TTL_HOURS` so product/legal can change without code deploy.

3.2 `Session` Model — Core Methods

```
# Single source of truth for candidate + assessor
def invite_status
  return 'revoked'   if invite_revoked?
  return 'expired'   if invite_expired?
  return 'completed' if invite_consumed?   # ended/failed
  status             # pending, active
end

def invite_usable?
  pending? && !invite_expired? && !invite_revoked?
end

def revoke_invite!
  update!(invite_revoked_at: Time.current)
end

# Token rotation — leaked link stays dead
def reissue_invite!
  update!(
    invite_token:          SecureRandom.hex(32),
    invite_expires_at:     self.class.invite_ttl_hours.hours.from_now,
    invite_revoked_at:     nil,
    invite_reissued_count: invite_reissued_count + 1
  )
end

def record_consent!
  update!(consent_given_at: Time.current) if consent_given_at.nil?
  consent_given_at
end

# Candidate-facing URL — MUST point to React app, not API
def invite_url
  base = ENV['WEB_BASE_URL'].presence || ENV['CANDIDATE_BASE_URL'].presence || 'http://localh
  "#{base.to_s.chomp('/')}/interview/#{invite_token}"
end
```

Key decisions:

- `invite_status` precedence: revoked > expired > consumed > native status.
- `reissue_invite!` **rotates** the token, does **not** extend. A leaked link must stay dead.
- `invite_url` reads `WEB_BASE_URL` (prod) → `CANDIDATE_BASE_URL` (compat) → `localhost:5173` (dev). Documented in `application.yml.sample`.

3.3 `SessionsController` — Candidate Endpoints

```
# GET /sessions/:token/candidate
def candidate
  session = Session.find_by!(invite_token: params[:token])
  # ... precise error codes
  render json: { ..., invite_status: session.invite_status, ... }
rescue ActiveRecord::RecordNotFound
  json_error("Invite not found", :not_found, code: "invite_not_found")
end

# POST /sessions/:token/consent
def consent
  session = Session.find_by!(invite_token: params[:token])
  return json_error("Link expired", :gone, code: "invite_expired") if session.invite_expired?
  return json_error("Link revoked", :gone, code: "invite_revoked") if session.invite_revoked?
  return json_error("Already completed", :gone, code: "invite_completed") if session.invite_c
  session.record_consent!
  render json: { consent_given_at: session.consent_given_at, consent_required: false }
end
```

Error codes returned:

Situation	HTTP	`code`
Token not found	404	`invite_not_found`
Expired	410	`invite_expired`
Revoked	410	`invite_revoked`
Completed	410	`invite_completed`
In progress	410	`invite_in_progress`
Network/unknown	0/5xx	`network` / `unknown`

Why 410 (Gone) for expired/revoked/completed/in-progress? Semantically “resource no longer available” — distinguishes from “never existed” (404). Frontend maps 410 + code → precise UI.

3.4 `SessionsController` — Assessor Endpoints

```
# POST /sessions/:id/revoke_invite
def revoke_invite
  return json_error("Cannot revoke in-progress interview", :unprocessable_entity) if @session.
  return json_error("Cannot revoke completed interview", :unprocessable_entity) if @session.en
  @session.revoke_invite!
  render json: { session: @session.as_json(invite_fields) }
end

# POST /sessions/:id/reissue_invite
def reissue_invite
  return json_error("Cannot reissue in-progress interview", :unprocessable_entity) if @session
  return json_error("Cannot reissue completed interview", :unprocessable_entity) if @session.e
  @session.reissue_invite!
  render json: { session: @session.as_json(invite_fields), invite_url: @session.invite_url }
end
```

Why 422 for live/completed? Prevents "hide a live interview by revoking it" — the monitor action must stay visible while the interview runs.

3.5 Tenant Isolation — Portfolio & Override

```
# Before (IDOR)
def set_portfolio
  @portfolio = Portfolio.find(params[:id])  # <-- global lookup
end

# After (tenant-scoped)
def set_portfolio
  @portfolio = Portfolio.joins(:session)
                        .find_by!(id: params[:id], sessions: { tenant_id: current_tenant_id }
end
```

Same pattern for `PortfolioSkillsController#set_portfolio_skill`. Cross-tenant returns **404** (not 403) so the API doesn't confirm foreign resources exist.

3.6 Auth — Assessor Role

```
# User model
ROLES = %w[user admin assessor].freeze          # was missing 'assessor'
ASSESSOR_ROLES = %w[admin assessor].freeze      # was only %w[admin]
```

Login (`AuthenticationController`) already accepted both; now model validations agree.

3.7 Rack::Attack — Test Mode

```
# config/initializers/rack_attack.rb
if Rails.env.test?
  Rack::Attack.cache.store = ActiveSupport::Cache::MemoryStore.new
end
```

No Redis required in CI.

4. Frontend Deep Dive

4.1 `candidateAccess.ts` — Pure Mapping Layer

```
export function resolveCandidateAccessError(status?: number, code?: string): CandidateAccessE
  switch (code) {
    case "invite_not_found": return "not_found";
    case "invite_expired":   return "expired";
    case "invite_revoked":   return "revoked";
    case "invite_completed": return "completed";
    case "invite_in_progress": return "in_progress";
  }
  // Fallback to HTTP status for older responses
  if (status === 404) return "not_found";
  if (status === 410) return "expired";
  if (!status || status === 0 || status >= 500) return "network";
  return "unknown";
}
```

Why separate file? Zero React deps → unit-testable in isolation. 10 Vitest cases cover every code path.

4.2 `CandidateAccessState.tsx` — The Honest Screen

```
// Before: .catch(() => setInterviewState("complete")) — LIED
// After: explicit mapping → each state has distinct copy + action

const copy = getCandidateAccessCopy(error, language);
// expired:  "This interview link has expired" + "Ask hiring team to reissue"
// revoked:  "This interview link was cancelled" + "Contact hiring team"
```

```
// completed:"You have already completed this interview" (ONLY real completion)
// in_progress: "Already in progress" + "Contact hiring team to rejoin safely"
// network:  "Could not reach service" + RETRY button (only state with retry)
```

Key invariant: `completed` copy is **only** reachable from a genuine `invite_completed` code. The test asserts: `expect(screen.getByText(/completed/i)).not.toBeInTheDocument()` when error is `expired`.

4.3 `ConsentGate.tsx` — Affirmative Opt-In

```
const [accepted, setAccepted] = useState(false);


  ☑ setAccepted(e.target.checked)} />
  {copy.checkbox}  {/* "I agree to the recording..." */}


  {copy.continue}
```

- Checkbox **must** be ticked (affirmative consent).
- `disabled` until `accepted === true`.
- Server call `POST /sessions/:token/consent` is idempotent; refused on dead links (410).
- EN/ID copy via `CONSENT_COPY` constant.

4.4 `InterviewPage.tsx` — State Machine

```
LOADING → (accessError) → CandidateAccessState
       → (consentRequired) → ConsentGate
       → (idle) → HardwareCheck → "Ready" banner → Start Interview
       → (connecting/active/reconnecting/draining_audio) → Voice UI
       → (complete) → "Interview Complete" (honest)
```

No fake completion. `audioComplete` retries until success (endpoint now always returns `ended: true` or error).

4.5 `LanguageToggle` + `useCandidateLanguage`

```
function detectLanguage(): CandidateLanguage {
  const stored = localStorage.getItem(STORAGE_KEY);
  if (stored === "en" || stored === "id") return stored;
```

```
  return navigator.language?.toLowerCase().startsWith("id") ? "id" : "en";
}
```

Persists to `localStorage`, sets `document.documentElement.lang`. Default `id` for Indonesian browser locale.

4.6 `AssessmentInvitePage.tsx` — Assessor Dashboard

- **Status chips** via `inviteStatusOf(session)` (reads `session.invite_status` from API).
- **Actions per state:**

State	Copy	Revoke	Reissue	Monitor	Results
pending	"Awaiting candidate"	YES	no	no	no
expired	"Invite expired"	no	YES	no	no
revoked	"Invite revoked"	no	YES	no	no
active	"Live"	no (422)	no	YES	no
completed	"Completed"	no	no	no	YES

- **Live-only polling:** `useEffect` checks `sessions.some(s => s.status === "active")` before starting 5 s interval.
- **Revoke confirmation dialog** explains candidate-visible consequence.
- **Reissue banner** shows new link + “Previous links stay invalid”.
- **Responsive rows:** `flex-col sm:flex-row`, header stacks on mobile, no horizontal overflow at 390 px.

5. Test Strategy

5.1 API — 43 RSpec Examples

Spec File	Examples	What It Proves
`session_spec.rb`	18	Defaults, URL origin, expiry/revocation/consumption, status precedence, token rotation, consent idempotency
`candidate_invites_spec.rb`	12	Every candidate code path, consent, idempotency, audio-complete behaviour

`invite_management_spec.rb`	10	Revoke/reissue rules, auth, role rejection, cross-tenant 404
`portfolio_tenant_isolation_spec.rb`	4	Read/write isolation for portfolios & overrides

Run: `cd api && bundle exec rspec` → `43 examples, 0 failures`

5.2 Web — 24 Vitest Examples

Spec File	Examples	What It Proves
`candidateAccess.test.ts`	10	Code→state mapping, status fallback, network classification, EN/ID copy
`CandidateAccessState.test.tsx`	4	Expired vs retryable rendering, Indonesian copy, **no false completion**
`ConsentGate.test.tsx`	3	Affirmative opt-in, language switch, error handling
`InterviewPage.test.tsx`	7	End-to-end state selection with mocked API + device check

Run: `cd web && npm test` → `24 passed` + `npm run build` (includes `tsc --noEmit`)

5.3 Seeded-Fault Proof (The "Tests Can Fail" Evidence)

Fault	Change	Result
Backend expiry removed	`def invite_usable?; pending? && !invite_revoked?; end`	**3 failures**: model spec, candidate lookup, consent refusal
Frontend expired→completed	`case "invite_expired": return "completed"`	**2 failures**: mapping test, rendering test

Logs committed to `assessment/evidence/seeded-fault-backend.txt` and `seeded-fault-frontend.txt`.

6. CI/CD — `.github/workflows/ci.yml`

```
jobs:
  api:
    runs-on: ubuntu-latest
```

```
    services:
      postgres:
        image: postgres:16
        ports: ["5432:5432"]
    env:
      RAILS_ENV: test
      DB_HOST: localhost
      DB_NAME: rakamin_test
      SECRET_KEY_BASE: ci-only-secret...
      REDIS_URL: redis://localhost:6379/1
      ALLOWED_ORIGINS: "*"          # ← fixes CORS initializer crash
    steps:
      - uses: actions/checkout@v4
      - uses: ruby/setup-ruby@v1
        with:
          ruby-version: "3.3.2"
          bundler-cache: true
          working-directory: api
      - run: bundle exec rails db:create db:schema:load
      - run: bundle exec rspec --format progress

  web:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: web
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm
          cache-dependency-path: web/package-lock.json
      - run: npm ci
      - run: npm test
      - run: npm run build
```

Pinned Ruby 3.3.2 to match `.ruby-version` / Gemfile (CI doesn't float on minors).

7. Trade-off Matrix (Option A vs B)

Criterion	Option A — Lifecycle columns on `sessions` (chosen)	Option B — Separate `invitations` aggregate + events
Time to safe release	High: 1 reversible migration	Low: new aggregate, backfill, dual-read/write

Backward compat	High: additive columns	Medium: every reader must switch
Token rotation	Simple: rotate `invite_token` + counter	Cleaner: each invitation row immutable
Audit/compliance	Adequate: timestamps + counter	Strong: append-only events with actor/reason
Multiple invites/candidate	Weak (1:1)	Strong (1:N)
Migration risk	Low	Higher
Test surface	Small, focused	Larger

Verdict: Ship Option A now. Adopt Option B when multiple invites, compliance-grade audit, or invite analytics become requirements.

8. Demo & Verification Checklist

```
# API
cd api
bundle install
cp config/application.yml.sample config/application.yml
bundle exec rails db:create db:migrate db:seed
bundle exec rails runner db/seeds_demo.rb   # ← deterministic 6 candidates
bundle exec rails s -p 3001

# Web (separate terminal)
cd web
npm install
cp .env.example .env   # VITE_API_BASE_URL=http://localhost:3001/api/v1
npm run dev            # http://localhost:5173
```

Assessor login: `assessor@example.test` / `demo-password-123` (dev seed only, overridable via `DEMO_ASSESSOR_PASSWORD`)

What to demo first:

1. `/assessments//invite` — six rows, each with correct chip + actions
2. Revoke Ahmad → confirm dialog → row shows revoked
3. Fresh link → consent gate (EN → Bahasa toggle) → hardware check
4. Expired / Revoked / Completed / In-progress / Not-found links — distinct copy
5. Network error → retry button

6. Reissue Siti → new token, old URL dead, banner says “Previous links stay invalid”
7. Mobile view (390×844) — no horizontal overflow

9. How to Explain Any Part in 30 Seconds

Question	30-Second Answer
Why rotate tokens on reissue?	A leaked link must stay dead. Extending the old token would keep the leak alive. Rotation gives a fresh 72 h window and increments a counter for audit.
Why 410 not 404 for expired?	404 = “never existed”. 410 = “existed, now gone”. Frontend distinguishes: expired shows “ask for reissue”; not-found shows “check the link”.
Why can’t I revoke a live interview?	Revocation would hide the “Monitor” action while the interview is still running. The assessor must explicitly end it, not revoke the invite.
How is consent auditable?	`consent_given_at` timestamp written server-side via idempotent `POST /consent`. Not a client flag — survives refresh, network loss, or malicious user.
How do you prevent cross-tenant portfolio access?	Every portfolio/override lookup joins through `sessions` and filters `sessions.tenant_id = current_tenant_id`. Cross-tenant returns 404 (not 403) so existence isn’t leaked.
Why no Redis in CI?	`Rack::Attack` uses `ActiveSupport::Cache::MemoryStore` in `test` env. Keeps CI fast and dependency-free.
What’s the seeded-fault proof?	I deliberately broke the two highest-risk behaviours (expiry check, expired→completed mapping) and the suites caught them (3 + 2 failures). Then restored. Proves tests assert the *product promise*, not just implementation.

10. Known Limitations (Honest & Documented)

Limitation	Why Not Fixed	Next Step
No resume flow for interrupted interview	Scope; honest “in-progress” state is safer than fake resume	Signed resume token scoped to live session
No email delivery	Assessor copies link manually	Mailer + bounce handling in Invitation Service
Expiry is read-time only	Correct for access control	Sweeper job to notify assessors before lapse
Consent copy needs legal review	Mechanism + timestamps done; wording is product/legal	UU PDP legal sign-off

Token in URL (referrer/history leak)	256-bit entropy + rotation mitigates	One-time exchange for short-lived cookie on first load
CORS `ALLOWED_ORIGINS: "*"` in sample	Dev only	Restrict before production
One invite per session	Simpler model	Move to `invitations` aggregate (Option B) when needed

11. File Map (What You Touched)

```
api/
├── db/
│   ├── migrate/20261106000000_add_invite_lifecycle_to_sessions.rb
│   └── seeds_demo.rb
├── app/
│   ├── models/session.rb                    # invite_usable?, invite_status, revoke!, reissue
│   ├── models/user.rb                       # ROLES, ASSESSOR_ROLES
│   ├── controllers/api/v1/
│   │   ├── sessions_controller.rb           # candidate codes, consent, revoke, reissue
│   │   ├── portfolios_controller.rb         # tenant-scoped lookup
│   │   └── portfolio_skills_controller.rb   # tenant-scoped lookup
│   └── controllers/concerns/response.rb     # json_error(..., code:)
├── config/
│   ├── application.rb                       # autoload_paths << (not +=)
│   ├── initializers/
│   │   ├── cors.rb                          # ENV.fetch("ALLOWED_ORIGINS") — fixed by CI env
│   │   ├── rack_attack.rb                   # MemoryStore in test
│   │   └── websocket.rb                     # require_relative .../_web_socket_middleware
│   └── application.yml.sample               # WEB_BASE_URL, CANDIDATE_BASE_URL documented
├── spec/
│   ├── models/session_spec.rb               # 18
│   ├── requests/candidate_invites_spec.rb   # 12
│   ├── requests/invite_management_spec.rb   # 10
│   └── requests/portfolio_tenant_isolation_spec.rb # 4
└── Gemfile                                  # ruby '3.3.2' (pinned)

web/
├── src/
│   ├── lib/candidateAccess.ts               # pure mapping + EN/ID copy
│   ├── lib/candidateAccess.test.ts          # 10
│   ├── components/candidate/
│   │   ├── CandidateAccessState.tsx         # honest screens
│   │   ├── CandidateAccessState.test.tsx    # 4
│   │   ├── ConsentGate.tsx                  # affirmative checkbox
```

```
│   │   ├── ConsentGate.test.tsx            # 3
│   │   └── LanguageToggle.tsx              # persisted EN/ID
│   ├── pages/
│   │   ├── interview/InterviewPage.tsx     # state machine
│   │   ├── interview/InterviewPage.test.tsx # 7
│   │   └── assessments/AssessmentInvitePage.tsx # assessor dashboard
│   └── test/setup.ts
├── package.json                             # vitest, tsc, build
└── .env.example                            # VITE_API_BASE_URL

.github/workflows/ci.yml                    # API + Web jobs

assessment/
├── report.html                             # full case-study report
├── Case-Study-Report.pdf                   # generated from report.html
├── Case-Study-Walkthrough.mp4              # 3:48 narrated video
├── evidence/
│   ├── rspec-passing.txt
│   ├── vitest-passing.txt
│   ├── api-verification.txt                # curl transcript (tokens redacted)
│   ├── seeded-fault-backend.txt
│   └── seeded-fault-frontend.txt
└── screenshots/01-13.png                   # deterministic captures
```

12. One-Liners for the "Why Did You…?" Questions

- **Why `invite_status` on the model, not the controller?** Single source of truth. Candidate page and assessor page both call the same method — no drift.
- **Why not Soft Delete for revocation?** Revocation is a *state*, not a deletion. We need the token, expiry, and reissue counter for audit and for the "reissue" action.
- **Why `invite_reissued_count`?** Audit trail: how many times was this link rotated? Product can later show "Link reissued 2×".
- **Why `consent_given_at` nullable?** Idempotent endpoint — second POST returns same timestamp. `nil` means "not yet given".
- **Why `ENV.fetch` with fallbacks in `invite_url`?** Works locally (`localhost:5173`), staging (`CANDIDATE_BASE_URL`), prod (`WEB_BASE_URL`) without code changes.
- **Why `404` for cross-tenant, not `403`?** `403` confirms "resource exists but you can't see it". `404` leaks nothing.
- **Why individual `<<` for autoload_paths?** Rails 7 freezes the array; `+=` mutates frozen → `FrozenError` .

- **Why rename `*_websocket_middleware.rb` → `*_web_socket_middleware.rb`?** Zeitwerk inflects `WebSocket` → `web_socket`; filename must match.
- **Why `MemoryStore` for Rack::Attack in test?** No Redis container in CI; in-memory is deterministic and fast.
- **Why seed 6 candidates?** Covers every invite state + portfolio/fit-gap + failed interview. One command = full demo.

13. Closing Statement (Use in Interview)

> "I chose the invite boundary because it's where **security** (token leakage, IDOR), **privacy** (expiry, consent, UU PDP), and **candidate trust** (honest states) collide. The baseline collapsed every failure into 'Interview Complete' — a lie. I made every state explicit, testable, and observable. The slice is narrow (one migration, 43 + 24 tests) but deep: I can defend every line, every trade-off, and every omission. The demo resets with one command, runs without a Gemini key, and the CI proves it stays green."

Generated from PR #240 — `feat/candidate-invite-trust` — ready for the technical interview.